Phase 7 Structured Note: Dividing Functions with Two Calls and Constant Work

1. Current Progress Analysis

Before Phase 7, you already learned how to turn recursive code into a recurrence relation and solve some common recurrence patterns.

You have already studied:

Phase	Main Idea	Example Recurrence	Final Result
Phase 1	Write a recurrence from recursive code	`T(n) = T(n - 1) + 1`	`O(n)`
Phase 2	Solve one-call decreasing recurrences	`T(n) = T(n - 1) + 1`	`O(n)`
Phase 3	Understand multiple decreasing calls	`T(n) = 2T(n - 1) + 1`	`O(2^n)`
Phase 4	Use the decreasing master theorem	`T(n) = aT(n - b) + f(n)`	Depends on `a`, `b`, and `f(n)`
Phase 5	Divide input by 2 with constant work	`T(n) = T(n / 2) + 1`	`O(log n)`
Phase 6	Divide input by 2 with linear work	`T(n) = T(n / 2) + n`	`O(n)`

The big idea from Phase 5 was:

```
If the input is divided by 2 each time, the number of levels is log n.
```

The big idea from Phase 6 was:

```
Even if there are log n levels, the total time also depends on the work done at
each level.
```

Now Phase 7 adds a new idea:

```
Instead of one recursive call on n / 2, the function makes two recursive calls
on n / 2.
```

So the main recurrence becomes:

```
T(n) = 2T(n / 2) + 1
```

The final result is:

```
T(n) = O(n)
```

---

2. Main Goal of Phase 7

The goal of Phase 7 is to understand recursive functions that:

1. Do constant work in the current call.
2. Make two recursive calls.
3. Give each recursive call half of the input.

The main code pattern is:

```
cout << n << " ";
algorithm(n / 2);
algorithm(n / 2);
```

The main recurrence is:

```
T(n) = 2T(n / 2) + 1
```

The final time complexity is:

```
O(n)
```

The main lesson is:

```
Two recursive calls do not always mean exponential time.
If each call receives half the input, the recursion tree becomes shallow.
For T(n) = 2T(n / 2) + 1, the total work is linear: O(n).
```

---

3. What Is a Dividing Function with Two Calls?

A dividing function is a recursive function where the input becomes smaller by division.

Example:

```
algorithm(n / 2);
```

This means the next call receives half of the current input.

In Phase 7, the function does this twice:

```
algorithm(n / 2);
algorithm(n / 2);
```

So one function call creates two smaller function calls.

Each smaller call receives:

```
n / 2
```

That is why the recurrence has:

```
2T(n / 2)
```

The `2` means there are two recursive calls.

The `n / 2` means each call gets half the input.

---

4. Main Code Example

```
#include <iostream>
using namespace std;
```

```cpp
void algorithm(int n) {
    if (n > 1) {
        cout << n << " ";      // constant work
        algorithm(n / 2);      // first recursive call
        algorithm(n / 2);      // second recursive call
    }
}

int main() {
    int n = 8;
    cout << "Output: ";
    algorithm(n);
    cout << endl;
    return 0;
}
```

---

5. What the Code Does

The function is:

```cpp
void algorithm(int n) {
    if (n > 1) {
        cout << n << " ";
        algorithm(n / 2);
        algorithm(n / 2);
    }
}
```

It does four main things:

1. It checks whether `n > 1`.
2. If true, it prints the current value of `n`.
3. It calls itself once with `n / 2`.
4. It calls itself again with `n / 2`.

The function stops when `n` becomes `1` or smaller.

---

6. Breaking the Code Into Parts

Part 1: Base Condition

```
if (n > 1)
```

This means the function continues only while `n` is greater than `1`.

When `n = 1`, the condition becomes false:

```
1 > 1 is false
```

So the function stops.

The base case is:

```
T(1) = 1
```

This means the stopping case takes constant time.

Part 2: Current Work

```
cout << n << " ";
```

This line prints one value.

Printing one value is constant work.

So the current work is:

```
+1
```

This is why Phase 7 is called constant-work recursion.

Part 3: First Recursive Call

```
algorithm(n / 2);
```

This call gives half of the input to the next recursive call.

Its time is:

```
T(n / 2)
```

Part 4: Second Recursive Call

```
algorithm(n / 2);
```

This is another recursive call with the same input size.

Its time is also:

```
T(n / 2)
```

So the two recursive calls together give:

```
T(n / 2) + T(n / 2)
```

That simplifies to:

```
2T(n / 2)
```

---

7. Writing the Recurrence Relation

The total time is:

```
first recursive call time + second recursive call time + current work
```

The first recursive call is:

```
T(n / 2)
```

The second recursive call is:

```
T(n / 2)
```

The current work is:

```
1
```

So:

```
T(n) = T(n / 2) + T(n / 2) + 1
```

Since the two recursive calls are the same size:

```
T(n) = 2T(n / 2) + 1
```

Base case:

```
T(1) = 1
```

Full recurrence:

```
T(n) = 2T(n / 2) + 1, for n > 1
T(1) = 1
```

---

8. Meaning of Each Part of the Recurrence

The recurrence is:

```
T(n) = 2T(n / 2) + 1
```

Part	Meaning
`T(n)`	Time taken for input size `n`
`2`	There are two recursive calls
`T(n / 2)`	Each recursive call receives half the input
`+1`	Constant work done in the current call

Part	Meaning
`T(1) = 1`	Base case takes constant time

In simple words:

```
The function does one small job now, then creates two smaller problems, each of
size n / 2.
```

---

9. General Division Recurrence Form

Dividing recurrences often follow this form:

```
T(n) = aT(n / b) + f(n)
```

Where:

Symbol	Meaning
`a`	Number of recursive calls
`b`	Division factor
`f(n)`	Extra work outside recursion

For Phase 7:

```
T(n) = 2T(n / 2) + 1
```

So:

Symbol	Value	Reason
`a`	`2`	There are two recursive calls
`b`	`2`	The input is divided by 2
`f(n)`	`1`	The function prints once, which is constant work

---

10. Dry Run for `algorithm(8)`

Call:

```
algorithm(8);
```

Step-by-step explanation

First call:

```
algorithm(8)
```

Since `8 > 1`, it prints:

```
8
```

Then it calls:

```
algorithm(4)
algorithm(4)
```

The first `algorithm(4)` prints:

```
4
```

Then it calls:

```
algorithm(2)
algorithm(2)
```

The first `algorithm(2)` prints:

```
2
```

Then it calls:

```
algorithm(1)
algorithm(1)
```

Both `algorithm(1)` calls stop because `1 > 1` is false.

Then the second `algorithm(2)` under the first `algorithm(4)` runs.

It prints:

```
2
```

Then it calls `algorithm(1)` twice, and both stop.

After the first `algorithm(4)` finishes, the second `algorithm(4)` runs.

It prints:

```
4
```

Then it calls two `algorithm(2)` functions.

Each `algorithm(2)` prints:

```
2
```

Then each one calls `algorithm(1)` twice and stops.

Final output:

```
8 4 2 2 4 2 2
```

---

11. Dry Run Table for `n = 8`

Call	Condition	Work Done	Next Calls
`algorithm(8)`	`8 > 1` true	print `8`	`algorithm(4)`, `algorithm(4)`

Call	Condition	Work Done	Next Calls
first `algorithm(4)`	`4 > 1` true	print `4`	`algorithm(2)`, `algorithm(2)`
first `algorithm(2)`	`2 > 1` true	print `2`	`algorithm(1)`, `algorithm(1)`
`algorithm(1)`	`1 > 1` false	stop	no calls
second `algorithm(2)`	`2 > 1` true	print `2`	`algorithm(1)`, `algorithm(1)`
second `algorithm(4)`	`4 > 1` true	print `4`	`algorithm(2)`, `algorithm(2)`
remaining `algorithm(2)` calls	`2 > 1` true	print `2` each	`algorithm(1)`, `algorithm(1)`

Output:

```
8 4 2 2 4 2 2
```

---

12. Recursion Tree Intuition

The recursive calls form a tree.

For `n = 8`, the tree looks like this:

```
            8
          /
        4       4
      /   \     /
    2     2   2     2
```

The `1` calls are not shown because they stop immediately.

Each node in the tree represents a working call.

The working calls are:

```
8
4 4
2 2 2 2
```

Counting the nodes:

```
1 + 2 + 4 = 7
```

That matches the output count:

```
8 4 2 2 4 2 2
```

There are 7 printed values.

---

13. Work by Level

For `algorithm(8)`:

Level	Input Size at Each Call	Number of Calls	Work per Call	Total Work at Level
Level 0	`8`	`1`	`1`	`1`
Level 1	`4`	`2`	`1`	`2`
Level 2	`2`	`4`	`1`	`4`
Stop level	`1`	`8`	`0` or constant stop	ignored in Big O

Total working-call work:

```
1 + 2 + 4 = 7
```

For a larger `n`, the pattern continues:

```
1 + 2 + 4 + 8 + ...
```

This is a geometric progression.

---

14. What Is a Geometric Progression?

A geometric progression, or GP series, is a sequence where each term is multiplied by the same number.

Example:

```
1, 2, 4, 8, 16, 32, ...
```

Each term is multiplied by `2`.

In Phase 7, the number of calls by level is:

```
1 + 2 + 4 + 8 + ... + 2^k
```

The formula for this sum is:

```
1 + 2 + 4 + ... + 2^k = 2^(k + 1) - 1
```

This formula is very important for solving Phase 7.

---

15. Solving the Recurrence by Successive Substitution

We solve:

```
T(n) = 2T(n / 2) + 1
```

Step 1: Start with the recurrence

```
T(n) = 2T(n / 2) + 1
```

Step 2: Substitute `T(n / 2)`

Using the same rule:

```
T(n / 2) = 2T(n / 4) + 1
```

Substitute it into the original recurrence:

```
T(n) = 2[2T(n / 4) + 1] + 1
```

Multiply:

```
T(n) = 4T(n / 4) + 2 + 1
```

Write `4` as `2^2`:

```
T(n) = 2^2T(n / 4) + 2 + 1
```

Step 3: Substitute again

Using the same rule:

```
T(n / 4) = 2T(n / 8) + 1
```

Substitute:

```
T(n) = 2^2[2T(n / 8) + 1] + 2 + 1
```

Multiply:

```
T(n) = 2^3T(n / 8) + 2^2 + 2 + 1
```

Step 4: See the pattern

After 1 step:

```
T(n) = 2T(n / 2) + 1
```

After 2 steps:

```
T(n) = 2^2T(n / 4) + 2 + 1
```

After 3 steps:

```
T(n) = 2^3T(n / 8) + 2^2 + 2 + 1
```

After `k` steps:

```
T(n) = 2^kT(n / 2^k) + 2^(k - 1) + 2^(k - 2) + ... + 2 + 1
```

This is the general pattern.

---

16. Reaching the Base Case

The base case is:

```
T(1) = 1
```

So we want the input inside `T(...)` to become `1`.

The recursive part is:

```
T(n / 2^k)
```

We want:

```
n / 2^k = 1
```

Now solve this equation.

Multiply both sides by `2^k`:

```
n = 2^k
```

So:

```
2^k = n
```

Taking logarithm base 2:

```
k = log₂(n)
```

This means the recursion has about $\log_2(n)$ levels.

---

17. Substitute Back

The pattern is:

```
T(n) = 2^kT(n / 2^k) + 2^(k - 1) + 2^(k - 2) + ... + 2 + 1
```

At the base case:

```
n / 2^k = 1
```

So:

```
T(n / 2^k) = T(1)
```

Then:

```
T(n) = 2^kT(1) + 2^(k - 1) + 2^(k - 2) + ... + 2 + 1
```

Since:

```
T(1) = 1
```

We get:

```
T(n) = 2^k + 2^(k - 1) + 2^(k - 2) + ... + 2 + 1
```

Rewrite it in increasing order:

```
T(n) = 1 + 2 + 4 + ... + 2^k
```

Use the GP sum formula:

```
1 + 2 + 4 + ... + 2^k = 2^(k + 1) - 1
```

So:

```
T(n) = 2^(k + 1) - 1
```

But we know:

```
2^k = n
```

So:

```
2^(k + 1) = 2 × 2^k
```

Since `2^k = n`:

```
2^(k + 1) = 2n
```

Therefore:

```
T(n) = 2n - 1
```

In Big O notation:

```
T(n) = O(n)
```

---

18. Full Solution in One Place

Start:

```
T(n) = 2T(n / 2) + 1
```

Substitute once:

```
T(n) = 2[2T(n / 4) + 1] + 1
T(n) = 2^2T(n / 4) + 2 + 1
```

Substitute again:

```
T(n) = 2^3T(n / 8) + 2^2 + 2 + 1
```

Pattern:

```
T(n) = 2^kT(n / 2^k) + 2^(k - 1) + ... + 2 + 1
```

Use the base case:

```
n / 2^k = 1
```

Solve:

```
n = 2^k
2^k = n
k = log₂(n)
```

Substitute into the pattern:

```
T(n) = 2^kT(1) + 2^(k - 1) + ... + 2 + 1
```

Since `T(1) = 1`:

```
T(n) = 2^k + 2^(k - 1) + ... + 2 + 1
```

This is:

```
1 + 2 + 4 + ... + 2^k
```

GP sum:

```
1 + 2 + 4 + ... + 2^k = 2^(k + 1) - 1
```

Since `2^k = n`:

```
2^(k + 1) = 2n
```

Final:

```
T(n) = 2n - 1
```

Big O:

```
T(n) = O(n)
```

---

19. Beginner-Friendly Meaning of the Solution

The recurrence:

```
T(n) = 2T(n / 2) + 1
```

means:

```
Do 1 small unit of work now, then solve two smaller problems.
Each smaller problem has size n / 2.
```

The recursion tree grows wider:

```
1 call
2 calls
4 calls
8 calls
...
```

But the input size shrinks quickly:

```
n
n / 2
n / 4
n / 8
...
1
```

Because the input is divided by 2 each time, there are only `log n` levels.

The total number of nodes in the tree becomes about:

```
2n - 1
```

So the final time complexity is:

```
O(n)
```

---

20. Why the Answer Is `O(n)`, Not `O(2^n)`

This is one of the most important points in Phase 7.

In Phase 3, you studied:

```
T(n) = 2T(n - 1) + 1
```

That became exponential because the input only decreased by `1`.

The input path looked like:

```
n -> n - 1 -> n - 2 -> n - 3 -> ... -> 0
```

That creates many levels.

The tree has many levels, and each level doubles.

So the time becomes:

```
O(2^n)
```

But in Phase 7, the recurrence is:

```
T(n) = 2T(n / 2) + 1
```

The input is divided by 2:

```
n -> n / 2 -> n / 4 -> n / 8 -> ... -> 1
```

This creates only `log n` levels.

So even though the tree doubles at each level, the tree does not get very deep.

Examples:

For `n = 8`:

```
1 + 2 + 4 = 7
```

For `n = 16`:

```
1 + 2 + 4 + 8 = 15
```

For `n = 32`:

```
1 + 2 + 4 + 8 + 16 = 31
```

These totals are close to:

```
2n
```

So the final time is:

```
O(n)
```

Memory sentence:

```
Two calls with n - 1 can be exponential.
Two calls with n / 2 can be linear.
```

---

21. Comparison: Phase 3 vs Phase 7

Feature	Phase 3	Phase 7
Recurrence	`T(n) = 2T(n - 1) + 1`	`T(n) = 2T(n / 2) + 1`
Number of recursive calls	2	2
Input change	subtracts 1	divides by 2
Shape	branching tree	branching tree
Number of levels	`n`	`log n`
Total time	`O(2^n)`	`O(n)`
Stack space	`O(n)`	`O(log n)`

The number of calls is the same: two recursive calls.

The difference is how the input changes.

In Phase 3:

```
n becomes n - 1
```

In Phase 7:

```
n becomes n / 2
```

That one difference changes the answer from exponential to linear.

---

22. Time Complexity

The time complexity is:

```
O(n)
```

Why?

Because the total work is:

```
1 + 2 + 4 + ... + 2^k
```

And because:

```
2^k = n
```

The sum becomes:

```
2n - 1
```

In Big O notation:

```
O(2n - 1) = O(n)
```

So:

```
Time complexity = O(n)
```

---

23. Stack Space Complexity

The stack space complexity is:

```
O(log n)
```

Why?

Even though the recursion creates many total calls, the computer does not keep all calls active at the same time.

The deepest active path looks like this:

```
algorithm(n)
algorithm(n / 2)
```

```
algorithm(n / 4)
algorithm(n / 8)
...
algorithm(1)
```

The length of this path is:

```
log₂(n)
```

So the maximum number of calls waiting on the stack at one time is:

```
O(log n)
```

Important difference:

```
Total calls = O(n)
Stack depth = O(log n)
```

---

24. Time Complexity vs Stack Space

For Phase 7:

Recurrence	Time Complexity	Stack Space
`T(n) = 2T(n / 2) + 1`	`O(n)`	`O(log n)`

Why time is `O(n)`:

```
The total number of working calls is about 2n - 1.
```

Why stack space is `O(log n)`:

```
The deepest path keeps dividing n by 2 until reaching 1.
```

---

25. Comparison with Phase 5 and Phase 6

Feature	Phase 5	Phase 6	Phase 7
Recurrence	`T(n) = T(n / 2) + 1`	`T(n) = T(n / 2) + n`	`T(n) = 2T(n / 2) + 1`
Recursive calls	1	1	2
Input change	`n / 2`	`n / 2`	`n / 2`
Extra work	constant	linear	constant
Main series	`1 + 1 + 1 + ...`	`n + n/2 + n/4 + ...`	`1 + 2 + 4 + ...`
Time complexity	`O(log n)`	`O(n)`	`O(n)`
Stack space	`O(log n)`	`O(log n)`	`O(log n)`

Key lesson:

```
All three phases divide the input by 2, so all have log n stack depth.
But their time complexities differ because the number of calls and extra work
differ.
```

---

26. Important Terms in Phase 7

Dividing Function

A recursive function where the input becomes smaller by division.

Example:

```
algorithm(n / 2);
```

Division Recurrence

A recurrence where the recursive part contains division.

Example:

```
T(n) = 2T(n / 2) + 1
```

Constant Work

Work that takes the same amount of time regardless of input size.

Example:

```
cout << n << " ";
```

This is written as:

```
+1
```

Two Recursive Calls

The function calls itself twice.

Example:

```
algorithm(n / 2);
algorithm(n / 2);
```

This gives:

```
2T(n / 2)
```

Base Case

The condition where recursion stops.

In Phase 7:

```
if (n > 1)
```

The function stops when:

```
n = 1
```

So:

```
T(1) = 1
```

Recursion Tree

A tree diagram that shows how recursive calls split.

For Phase 7:

```
              n
           /
        n/2         n/2
       /    \      /
    n/4    n/4 n/4    n/4
```

Geometric Progression

A series where each term is multiplied by the same number.

In Phase 7:

```
1 + 2 + 4 + 8 + ...
```

The sum is:

```
2^(k + 1) - 1
```

Recursion Depth

The length of the longest path from the first call to the base case.

For Phase 7:

```
n -> n/2 -> n/4 -> n/8 -> ... -> 1
```

So the depth is:

```
O(log n)
```

---

27. Complete Coding Program

```
#include <iostream>
using namespace std;

// This function prints n, then calls itself twice with n / 2.
void algorithm(int n) {
    if (n > 1) {
        cout << n << " ";      // constant work: O(1)
        algorithm(n / 2);      // first recursive call on half input
        algorithm(n / 2);      // second recursive call on half input
    }
}

int main() {
    int n = 8;
    cout << "Output: ";
    algorithm(n);
    cout << endl;
    return 0;
}
```

Output for `n = 8`:

```
Output: 8 4 2 2 4 2 2
```

Recurrence:

```
T(n) = 2T(n / 2) + 1
T(1) = 1
```

Time complexity:

```
O(n)
```

Stack space:

```
O(log n)
```

---

28. Common Beginner Mistakes

Mistake 1: Thinking Two Recursive Calls Always Mean Exponential Time

Wrong idea:

```
There are two recursive calls, so the answer must be O(2^n).
```

Correct idea:

```
Two calls with n - 1 can be exponential.
Two calls with n / 2 can be linear.
```

Why?

Because dividing by 2 creates only `log n` levels.

Mistake 2: Confusing Phase 3 with Phase 7

Phase 3:

```
T(n) = 2T(n - 1) + 1
```

Phase 7:

```
T(n) = 2T(n / 2) + 1
```

They look similar, but they are very different.

The first one subtracts.

The second one divides.

That changes the result.

Mistake 3: Forgetting the Current Work

Some students write:

```
T(n) = 2T(n / 2)
```

But the function also prints once:

```
cout << n << " ";
```

So the correct recurrence is:

```
T(n) = 2T(n / 2) + 1
```

Mistake 4: Forgetting the Base Case

The function stops when:

```
n = 1
```

So the base case is:

```
T(1) = 1
```

Without the base case, we do not know when to stop substitution.

Mistake 5: Using the Wrong Base Case Equation

For decreasing recurrences, you used:

```
n - k = 0
```

But Phase 7 is a dividing recurrence.

So you must use:

```
n / 2^k = 1
```

This gives:

```
2^k = n
```

and:

```
k = log₂(n)
```

Mistake 6: Not Replacing `2^k` with `n`

In the solution, we find:

```
2^k = n
```

This is very important.

It lets us simplify:

```
2^(k + 1) - 1
```

into:

```
2n - 1
```

Then:

```
T(n) = O(n)
```

Mistake 7: Thinking Stack Space Is Also `O(n)` Because Time Is `O(n)`

The time is:

```
O(n)
```

But stack space is:

```
O(log n)
```

Why?

Because the deepest active path is:

```
n -> n/2 -> n/4 -> ... -> 1
```

That has only `log n` calls.

---

29. Step-by-Step Checklist for Phase 7 Problems

When you see a Phase 7-style recursive function, follow these steps.

Step 1: Find the base case

Look for the stopping condition.

Example:

```
if (n > 1)
```

The function stops when:

```
n = 1
```

So:

```
T(1) = 1
```

Step 2: Count recursive calls

Example:

```
algorithm(n / 2);
algorithm(n / 2);
```

There are two recursive calls.

So:

```
a = 2
```

Step 3: Track how the input changes

The input changes from:

```
n
```

to:

```
n / 2
```

So the recursive part is:

```
2T(n / 2)
```

Step 4: Count extra work

Example:

```
cout << n << " ";
```

This prints once.

So the extra work is:

```
+1
```

Step 5: Write the recurrence

```
T(n) = 2T(n / 2) + 1
T(1) = 1
```

Step 6: Substitute repeatedly

```
T(n) = 2T(n / 2) + 1
T(n) = 2^2T(n / 4) + 2 + 1
T(n) = 2^3T(n / 8) + 2^2 + 2 + 1
```

Step 7: Find the pattern

```
T(n) = 2^kT(n / 2^k) + 2^(k - 1) + ... + 2 + 1
```

Step 8: Use the base case

```
n / 2^k = 1
```

Step 9: Solve for `k`

```
n = 2^k
2^k = n
k = log₂(n)
```

Step 10: Use the GP sum

```
1 + 2 + 4 + ... + 2^k = 2^(k + 1) - 1
```

Step 11: Replace `2^k` with `n`

```
2^(k + 1) = 2 × 2^k = 2n
```

So:

```
T(n) = 2n - 1
```

Step 12: Write Big O

```
T(n) = O(n)
```

Stack space:

```
O(log n)
```

---

30. Mini Practice Example 1

Find the recurrence and time complexity.

```
void test(int n) {
    if (n > 1) {
        cout << "*";
        test(n / 2);
        test(n / 2);
    }
}
```

Solution

Base case:

```
T(1) = 1
```

There are two recursive calls:

```
test(n / 2);
test(n / 2);
```

So the recursive part is:

```
2T(n / 2)
```

The current work is printing once:

```
+1
```

Recurrence:

```
T(n) = 2T(n / 2) + 1
```

Time complexity:

```
O(n)
```

Stack space:

```
O(log n)
```

---

31. Mini Practice Example 2

Find the recurrence and time complexity.

```
void divideTwice(int n) {
    if (n > 1) {
        cout << n;
        divideTwice(n / 2);
        divideTwice(n / 2);
    }
}
```

Solution

The function stops when:

```
n = 1
```

So:

```
T(1) = 1
```

The function has two recursive calls.

Each recursive call receives:

```
n / 2
```

So the recursive part is:

```
2T(n / 2)
```

The function prints once, so the extra work is:

```
+1
```

Final recurrence:

```
T(n) = 2T(n / 2) + 1
T(1) = 1
```

Final time:

```
O(n)
```

Stack space:

```
O(log n)
```

---

32. Mini Practice Example 3

Find the recurrence only.

```
void example(int n) {
    if (n > 1) {
        cout << "Hello";
        example(n / 2);
        example(n / 2);
    }
}
```

Solution

There are two recursive calls:

```
2T(n / 2)
```

There is constant work:

```
+1
```

Base case:

```
T(1) = 1
```

Recurrence:

```
T(n) = 2T(n / 2) + 1
T(1) = 1
```

Time complexity:

```
O(n)
```

---

33. Simple Way to Remember Phase 7

For this code pattern:

```
function(n) {
    if (n > 1) {
        do one small work;
        function(n / 2);
        function(n / 2);
    }
}
```

The recurrence is:

```
T(n) = 2T(n / 2) + 1
```

The time complexity is:

```
O(n)
```

The stack space is:

```
O(log n)
```

Memory trick:

```
Two half-size calls plus constant work gives linear time.
```

Another memory trick:

```
T(n) = 2T(n / 2) + 1  →  O(n)
```

Most important memory sentence:

```
Two calls do not automatically mean O(2^n).
Always check how the input changes.
```

---

34. Why Phase 7 Prepares You for Phase 8

Phase 7 is:

```
T(n) = 2T(n / 2) + 1
```

This means:

```
Two half-size recursive calls plus constant work.
```

The answer is:

```
O(n)
```

In Phase 8, the recurrence becomes:

```
T(n) = 2T(n / 2) + n
```

This means:

```
Two half-size recursive calls plus linear work.
```

That answer will be:

```
O(n log n)
```

So Phase 7 teaches you the recursion tree shape before Phase 8 adds heavier work at each level.

---

35. Final Phase 7 Summary

In Phase 7, you learned about dividing recursive functions with two recursive calls and constant work.

The main code was:

```
void algorithm(int n) {
    if (n > 1) {
        cout << n << " ";
        algorithm(n / 2);
        algorithm(n / 2);
    }
}
```

The function:

1. Prints the current value of `n`.
2. Calls itself with `n / 2`.
3. Calls itself again with `n / 2`.
4. Stops when `n = 1`.

The recurrence is:

```
T(n) = 2T(n / 2) + 1
T(1) = 1
```

Using substitution:

```
T(n) = 2T(n / 2) + 1
T(n) = 2^2T(n / 4) + 2 + 1
T(n) = 2^3T(n / 8) + 2^2 + 2 + 1
```

Pattern:

```
T(n) = 2^kT(n / 2^k) + 2^(k - 1) + ... + 2 + 1
```

Base case equation:

```
n / 2^k = 1
```

Solve:

```
2^k = n
k = log₂(n)
```

The work becomes a GP series:

```
1 + 2 + 4 + ... + 2^k
```

The GP sum is:

```
2^(k + 1) - 1
```

Since:

```
2^k = n
```

Then:

```
2^(k + 1) = 2n
```

So:

```
T(n) = 2n - 1
```

Final time complexity:

```
O(n)
```

Stack space:

```
O(log n)
```

The most important lesson is:

```
T(n) = 2T(n / 2) + 1 is linear, not exponential.
The input is divided by 2, so the recursion tree has only log n levels.
```

---

36. Key Formula From Phase 7

```
T(n) = 2T(n / 2) + 1
```

Base case:

```
T(1) = 1
```

Final time:

```
O(n)
```

Stack space:

```
O(log n)
```

Memory sentence:

```
Two recursive calls on half input plus constant work gives O(n).
```